5

Modelo Relacional: Tópicos Avançados

Especialização

A transformação de uma especialização do modelo de ER para o modelo relacional pode ser feita de 3 diferentes modos:

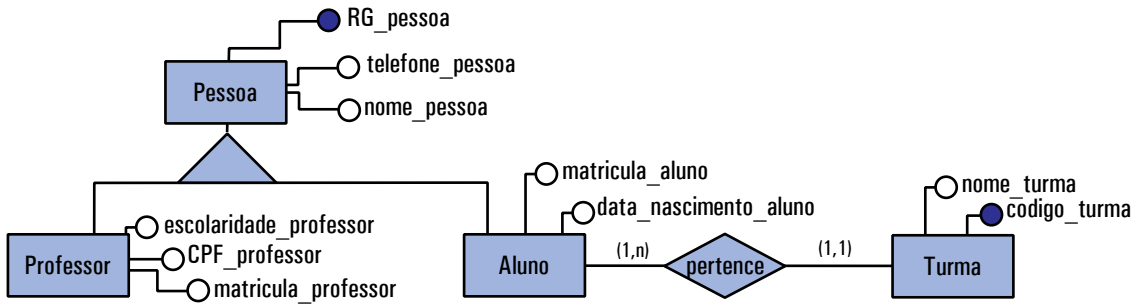
1. Criando uma tabela apenas para a entidade pai;
2. Criando tabelas apenas para as entidades filhas;
3. Criando uma tabela para cada entidade (tanto para a entidade pai, quanto para as filhas).

Na primeira situação, será criada uma tabela única com o nome da entidade pai e essa tabela irá conter: todos os atributos das entidades pai (genérica), os atributos da(s) entidade(s) filha(s) (entidades especializadas), atributos referentes a possíveis relacionamentos e um atributo chamado "tipo" que identificará qual entidade especializada está sendo representada em uma linha. A chave primária dessa tabela será a própria chave primária da entidade pai (ver figura 5.1).

Na segunda situação, serão criadas tabelas apenas para as entidades filhas. Cada entidade filha que virar uma tabela terá como atributos tantos os seus atributos específicos e de seus relacionamentos diretos, quanto os atributos da entidade pai, mais os atributos dos relacionamentos de outras entidades com a entidade pai. A chave primária de cada uma das tabelas especializadas será a chave primária da entidade pai, como mostra a figura 5.1. As tabelas criadas serão completamente independentes umas das outras.

Na terceira situação, serão criadas tabelas para todas as entidades (pai e filhas). Cada tabela terá seus atributos específicos, e os atributos dos seus relacionamentos. As tabelas referentes às entidades filhas também receberão como chave estrangeira a chave primária da entidade pai.

A chave primária para cada entidade filha será a chave estrangeira, que neste caso terá as duas funções (**PK** e **FK**). Caso exista algum atributo que identifique unicamente a entidade filha, ele poderá ser escolhido como chave primária e a chave primária da entidade pai passa apenas como chave estrangeira, como mostra a figura 5.1.



1º modo de conversão: Criando uma única tabela para a especialização, teremos:

```
tbTurma(codigo_turma, nome_turma)

tbPessoa(RG_pessoa, telefone_pessoa, nome_pessoa, matricula_aluno, data_
nascimento_aluno,codigo_turma,escolaridade_professor,CPF_professor,
matricula_professor, tipo_pessoa)
codigo_turma referencia tbTurma
```

2º modo de conversão: Criando tabelas apenas para as entidades filhas (especializadas), teremos:

```
tbTurma(codigo_turma, nome_turma)

tbAluno(RG_pessoa,nome_pessoa,telefone_pessoa,matricula_aluno,data_nascimento_
aluno, codigo_turma)
codigo_turma referencia tbTurma

tbProfessor(RG_pessoa, nome_pessoa, telefone_pessoa, matricula_professor,
CPF_professor, escolaridade_professor)
```

3º modo de conversão: Criando tabelas apenas para as entidades pai e filhas, teremos:

```
tbTurma(codigo_turma, nome_turma)

tbPessoa(RG_pessoa, nome_pessoa, telefone_pessoa)

tbProfessor(matricula_professor,RG_pessoa,CPF_professor,escolaridade_professor)
RG_pessoa referencia tbPessoa
tbAluno(matricula_aluno, RG_pessoa, data_nascimento_aluno, codigo_turma)
codigo_turma referencia tbTurma
RG_pessoa referencia tbPessoa
```

Figura 5.1 – Conversão de especialização para o modelo relacional

Na descrição do modelo relacional da figura 5.1, para o 3º modo de conversão, observe que foi escolhido como atributo chave para as tabelas especializadas (tbAluno e tbProfessor) os atributos matricula_aluno e matricula_professor respectivamente. O atributo RG_pessoa funcionará apenas como chave estrangeira, uma vez que temos um atributo em cada tabela especializada que identifica uma única linha da tabela. Caso não existisse tal atributo, a chave primária seria o próprio RG_pessoa que seria chave estrangeira e primária ao mesmo tempo. A chave primária não será composta nessa situação.

A pergunta que aparece nesse momento é a seguinte: qual abordagem eu devo utilizar quando o modelo apresenta uma especialização?

A primeira abordagem irá conter muitos valores nulos, uma vez que dado o tipo do objeto somente os atributos referentes àquele objeto serão preenchidos. Por isso, nem todos os atributos serão obrigatórios. Por outro lado, essa primeira abordagem tem a vantagem de dispensar a necessidade de junção entre tabelas, uma vez que os dados estão todos na mesma tabela.

A segunda abordagem é pouco recomendada, porque pode gerar redundância de dados, uma vez que os dados da entidade genérica são repetidos em todas tabelas especializadas. Assim, se uma pessoa for tanto professor como aluno, teremos as informações referentes a essa pessoa repetida nas duas tabelas. Portanto, essa abordagem só deve ser utilizada quando tivermos uma especialização exclusiva, ou seja, uma pessoa ou é do tipo aluno ou do tipo professor.

A terceira abordagem tem a vantagem de evitar os valores nulos que aparecem na primeira abordagem e ainda a de não permitir a duplicidade como na segunda abordagem.

Assim, o desenvolvedor da base de dados deve analisar todos os aspectos referentes à situação que se está modelando e optar pela solução que seja mais adequada ao problema.

Diagrama do Modelo Relacional

O modelo relacional pode ser descrito, como fizemos nos exemplos anteriores, ou pode ser diagramado (forma mais comum).

No Diagrama do Modelo Relacional, tudo que virou tabela, aplicando-se as regrinhas de conversão entre modelos, será representado por um retângulo. Esse retângulo irá conter o nome da tabela, seus atributos, os tipos dos atributos, a identificação da chave primária, a identificação da chave estrangeira e a cardinalidade do modelo.

A cardinalidade é atribuída considerando-se o Modelo de ER. Cardinalidade do tipo 1:1 e 1:N são representadas da mesma forma que no modelo de ER. Já a cardinalidade N:N não aparece no diagrama do modelo relacional, uma vez que o relacionamento N:N virou uma tabela. Portanto, todo relacionamento N:N dará origem a dois relacionamentos do tipo 1:N.

O exemplo da figura 5.2 apresenta o Diagrama do Modelo Relacional para o Diagrama de ER da figura 3.7. A ferramenta **brModelo** chama esse diagrama de Modelo Lógico (ou Esquema Lógico).

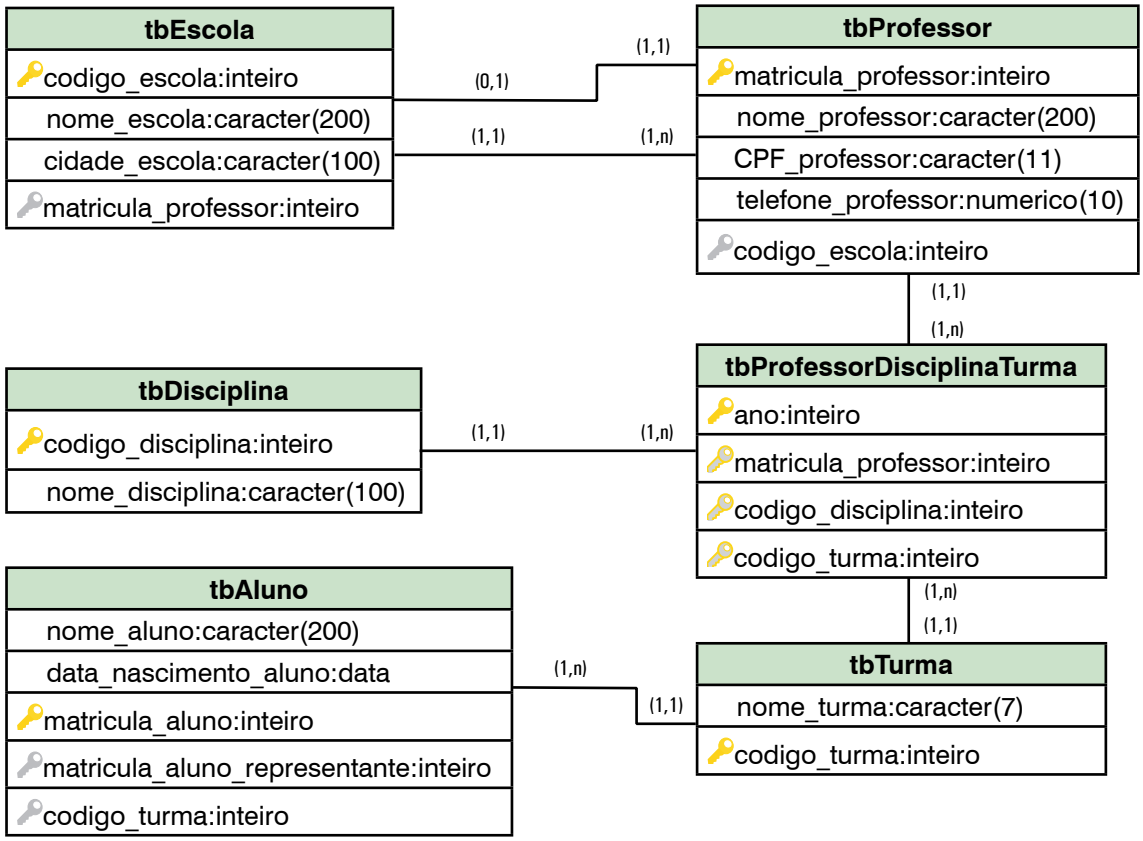


Figura 5.2 – Diagrama do modelo relacional para o diagrama de ER da figura 3.7

A chave primária nesse diagrama é indicada pela chave dourada, mas também poderia ser indicada grifando-se os atributos que formam a chave primária. A chave estrangeira é indicada pela chave de cor prata, mas também poderia ser indicada pelo asterisco. Aqueles atributos que são chaves estrangeiras e também ajudam a formar a chave primária, como mostrado na tabela tbProfessorDisciplinaTurma, são indicados pela chave dourada com o interior prateado.

Dicionário de Dados da Base de Dados

Após o modelo relacional ter sido descrito ou diagramado, é necessário criar o Dicionário de Dados para a base de dados.

O Dicionário de Dados da Base de Dados tem por objetivo descrever as propriedades de uma tabela, sua estrutura física e as restrições que cada atributo possui. Assim, o desenvolvedor que irá implementar o banco de dados saberá exatamente como a base deve ser criada.

No Dicionário de Dados da Base de Dados, cada tabela do modelo relacional deverá ser descrita e deverá conter os seguintes campos: Nome do Atributo, Descrição do Atributo, Tamanho, Tipo e Restrições (Valor Nulo, Regra de Domínio, Chaves, Valor *Default* e *Unique*).

A figura 5.3 apresenta um exemplo do Dicionário de Dados para a tbAluno. Foram acrescentados alguns atributos nessa tabela para que seja possível exemplificar cada uma das restrições citadas anteriormente.

Nome	Descrição	Tipo	Tamanho	Nulo	Regra (check)	Chave	Default	Unique
matricula_aluno	Armazena a matricula do aluno	Numérico	5	Não	__	PK	__	Não
RG_aluno	Armazena o RG do aluno	Caracter	11	Não	__	__	__	Sim
nome_aluno	Armazena o nome do aluno	Caracter	100	Não	__	__	__	Não
data_nascimento_aluno	Armazena a data de nascimento do aluno	Data	__	Não	__	__	__	Não
cidade_aluno	Armazena a cidade em que o aluno mora	Caracter	20	Sim	__	__	Curitiba	Não
matricula_aluno_representante	Armazena a matricula do aluno represen-tante	Numérico	5	Sim	__	__	__	Não
codigo_turma	Armazena o código da turma do aluno	Inteiro	__	Não	__	FK que referencia tbTurma	__	Não
sexo_aluno	Armazena o sexo a que o aluno pertence	Caracter	1	Não	M – Masculino F – Feminino	__	__	Não

Figura 5.3 – Dicionário de dados para a tabela tbAluno

Para alguns tipos de dados, não é possível definirmos o tamanho, como por exemplo o tipo Data, porque esses tipos já têm tamanho pré-definido pelo SGBD. As restrições aplicadas a um atributo definem as propriedades desse atributo.

A restrição de **Nulo** define se um atributo permite ou não o valor nulo, ou seja, define se o atributo será obrigatório ou não.

Uma restrição de **Domínio** ou **Regra de Domínio** define quais valores serão permitidos cadastrar para um atributo. No exemplo da figura 5.3, temos uma regra de domínio que diz que os valores permitidos para sexo são apenas "M" ou "F".

As restrições de **chave** permitem identificar a chave primária (**PK**) e as chaves estrangeiras (**FK**). É interessante que na definição da chave estrangeira também seja identificado a que tabela ela referencia.

A restrição de ***default*** permite que seja inserido um valor padrão caso o usuário não digite nada para o campo. No nosso exemplo, definiu-se que se o usuário não digitar nada para o campo cidade_aluno, o próprio SGBD armazena o valor "Curitiba" para esse campo.

A última restrição é a de **unicidade**. Essa restrição é aplicada apenas para atributos que não são chave primária e que não podem se repetir. No exemplo da figura 5.3, o atributo RG_aluno não é chave primária e não pode se repetir. Sendo assim, pode-se definir o atributo como *unique* (único). É redundante dizer que uma chave primária é *unique*, já que ela não se repete.

Normalização

O processo de normalização geralmente é aplicado quando temos uma base de dados que foi criada antes da existência de um banco de dados relacional ou foi desenvolvida sem considerar a existência de um banco de dados relacional.

Na maioria dos casos, esses sistemas são antigos e seus arquivos de dados possuem muitas informações redundantes e inconsistentes. Além disso, normalmente não existe nenhum documento que especifique o modelo de dados, o que torna muito difícil a manutenção desses sistemas ou as migrações para um banco de dados relacional.



Vale ressaltar que o modelo conceitual da base de dados agiliza bastante o processo de manutenção, porque permite que pessoas que não participaram do desenvolvimento do projeto possam entendê-lo mais rapidamente.

Às vezes, também encontramos sistemas não muito antigos e que, no entanto, foram implementados usando uma única tabela. Nesses casos, as informações estarão repetidas, haverá muitos valores nulos na base e será muito difícil executar consultas que retornem valores consistentes com o esperado.

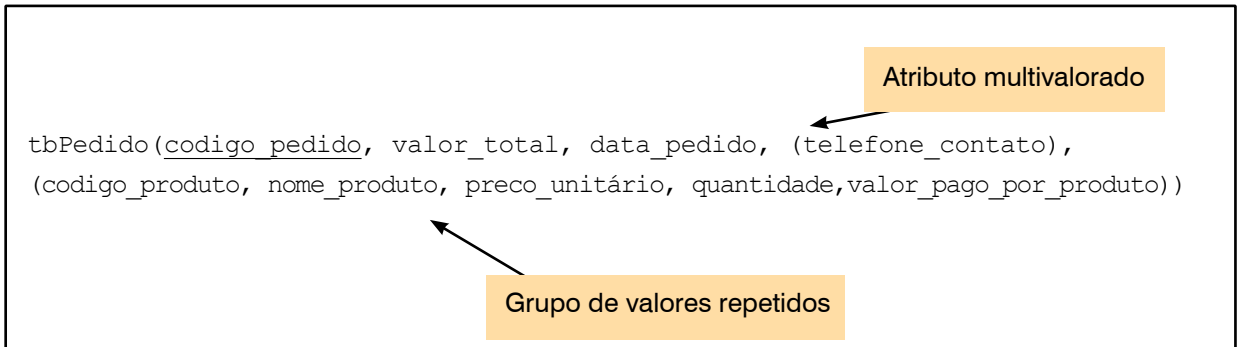
Para resolver ou minimizar os problemas apresentados, pode-se fazer o que chamamos de engenharia reversa do projeto, utilizando para isso o processo de normalização. A engenharia reversa do projeto consiste em, a partir dos dados armazenados, obter um modelo conceitual da base de dados e eliminar as redundâncias.

Também podemos utilizar o processo de normalização para conferir se o nosso modelo de dados está normalizado e, caso não esteja, pode-se normalizá-lo antes da implementação da base de dados no SGBD.

O processo de normalização consiste em um conjunto de regras, denominadas formas normais. A literatura apresenta 6 formas normais: 1FN, 2FN, 3FN, 4FN, 5FN e a de Boyce/Codd. Alguns autores tratam a forma normal de Boyce/Codd como sendo um caso específico da 3FN.

No entanto, para a maioria das bases de dados, a aplicação até a 3FN é suficiente. Sendo assim, só iremos abordar nesse livro as três primeiras formas normais.

A tabela tbPedido da figura 5.4 será utilizada para demonstrar o processo de normalização.



O modelo relacional acima poderia ser escrito como mostra a tabela:

Código pedido	Valor Pedido	Data pedido	Telefone de contato	Produto				
				Codigo produto	Nome produto	Valor unitário do produto	Quantidade	Valor pago por produto
100	3300,00	30/11/2009	2222-2222 9999-9999	1	Computador	1500,00	1	1500,00
				5	Impressora	600,00	2	1200,00
				6	Papel A4	12,00	50	600,00
101	3800,00	15/12/2009	2121-2121 9191-9191	2	Mouse	30,00	10	300,00
				5	Impressora	600,00	5	3000,00
				7	Teclado	50,00	10	500,00

Figura 5.4 – Exemplo de uma tabela não normalizada

Primeira Forma Normal (1FN)

Toda tabela está na 1FN se os seus atributos forem atômicos. Isso significa que não serão permitidos atributos compostos, multivalorados ou grupos repetidos de dados (também conhecidos como tabelas aninhadas).

Os grupos repetidos de dados ocorrem quando uma tabela aparece dentro de outra tabela. Por exemplo, na tabela tbPedido da figura 5.4, temos uma única tabela que armazena os dados de pedidos e de produtos. Assim, os dados referentes ao produto pertencem a uma tabela que está dentro da tabela pedido.

É possível perceber que a tabela tbPedido não está na 1FN porque além de grupos repetidos ela também tem um atributo multivalorado.

Para deixar uma tabela na 1FN, é necessário fazer o seguinte:

- Os atributos compostos devem ser decompostos e armazenados como atributos simples.
- Para cada atributo multivalorado será criada uma tabela que irá conter o atributo multivalorado mais a chave primária da tabela inicial, que passa como chave estrangeira. A chave primária da nova tabela será composta.
- Para cada grupo repetido será criada uma tabela que irá conter os atributos do grupo repetido mais a chave primária da tabela inicial, que passa como chave estrangeira e irá ajudar a compor a chave primária.

A figura 5.5 mostra como ficaria o modelo da figura 5.4 após aplicarmos a 1FN.

```
tbPedido(codigo_pedido, valor_total, data_pedido)

tbTelefoneContatoPedido(codigo_pedido, telefone_contato)
codigo_pedido referencia tbPedido

tbPedidoProduto(codigo_pedido, codigo_produto, nome_produto, preco_unitario,
quantidade, valor_pago_por_produto)

codigo_pedido referencia tbPedido
```

Figura 5.5 – Aplicação da 1FN para eliminar grupo repetido e atributo multivalorado

Segunda Forma Normal (2FN)

A 2FN só é aplicável para tabelas que possuem uma chave primária composta e que, além disso, tenham outros atributos que não façam parte da chave primária.

Uma tabela está na 2FN se estiver na 1FN e todo atributo que não compõe a chave primária deve ter dependência funcional total em relação à chave primária.

> No modelo relacional, a **dependência funcional** entre dois atributos, A e B, ocorre quando, em todas as linhas da tabela, para cada valor de A irá aparecer sempre o mesmo valor de B. Por exemplo, na tabela da figura 5.4 sempre que aparecer o código de produto "5" teremos como nome do produto "Impressora". Assim, o nome do produto depende funcionalmente do código do produto.
>
> Para denotar a dependência funcional, usa-se uma expressão na forma "codigo_produto → nome_produto". Isso significa que o nome do produto depende funcionalmente do código do produto.

Para que a dependência funcional seja total, o atributo não chave deve depender de toda a chave primária composta. Por exemplo, no caso da tabela tbPedidoProduto o atributo "quantidade" representa a quantidade de um produto que foi solicitada em um pedido. Assim, o atributo "quantidade" depende tanto do codigo_produto quanto do codigo_pedido, caracterizando uma **dependência funcional total**. O atributo nome_produto, por outro lado, tem uma **dependência funcional parcial**, porque depende apenas de parte da chave primária, ou seja, depende apenas do codigo_produto.

Para deixarmos o modelo na 2FN devemos:

- Repetir as tabelas que tenham chave primária simples, pois já estão na 2FN.
- Repetir as tabelas que tenham chave primária composta, mas que não tenham nenhum outro atributo além dos que compõem a chave primária.

- Para cada tabela que tiver chave primária composta e pelo menos um atributo que não faz parte da chave, deve-se verificar se cada um dos atributos não chave têm **dependência funcional total**. Caso a dependência não seja total, deve-se criar uma tabela com o atributo que depende parcialmente, mais o atributo do qual ele depende (que será chave primária na nova tabela e será chave estrangeira na tabela inicial).

A tabela 5.6 mostra como ficaria o modelo da figura 5.4 após aplicarmos a 2FN.

```
tbPedido(codigo_pedido, valor_total, data_pedido)

tbTelefoneContatoPedido(codigo_pedido, telefone_contato)
codigo_pedido referencia

tbPedidoProduto(codigo_pedido, codigo_produto, quantidade,
valor_pago_por_produto)
codigo_pedido referencia tbPedido
codigo_produto referencia tbProduto

tbProduto( codigo_produto, nome_produto, preco_unitario)
```

Figura 5.6 – Aplicação da 2FN para eliminar redundâncias

Terceira Forma Normal (3FN)

Uma tabela está na 3FN se estiver na 2FN e ela não possuir dependências transitivas.

> Dependência Transitiva ocorre quando existe um atributo que não é chave e nem faz parte da chave, mas que identifica outros atributos. Ou seja, existe um atributo não chave que depende de outro atributo não chave.

Como no modelo da figura 5.6 não existem dependências transitivas, ele já está na 3FN. No entanto, para entendermos melhor dependências transitivas, vamos supor a situação da figura 5.7.

```
tbDepartamento(codigo_depto, nome_depto, codigo_gerente, nome_gerente)
```

Figura 5.7 – Tabela com dependência transitiva

A tabela da figura 5.7 está na 1FN porque todos os seus atributos são atômicos e está na 2FN porque não tem chave primária composta. No entanto, temos uma dependência transitiva que ocorre com o atributo nome_gerente que depende do atributo codigo_gerente que não é chave e nem faz parte da chave primária.

Para eliminar dependências transitivas, deve-se criar uma nova tabela que irá conter o atributo que depende (ex.: nome_gerente) mais o atributo do qual ele é dependente (ex.: codigo_gerente). A figura 5.8 mostra como ficaria o modelo.

```
tbDepartamento(codigo_depto, nome_depto, codigo_gerente)
codigo_gerente referencia tbGerente

tbGerente (codigo_gerente, nome_gerente)
```

Figura 5.8 – Aplicação da 3FN para eliminar dependência transitiva

É interessante sempre aplicarmos as três formas normais ao nosso modelo relacional antes de implementar esse modelo no SGBD.



Atividades

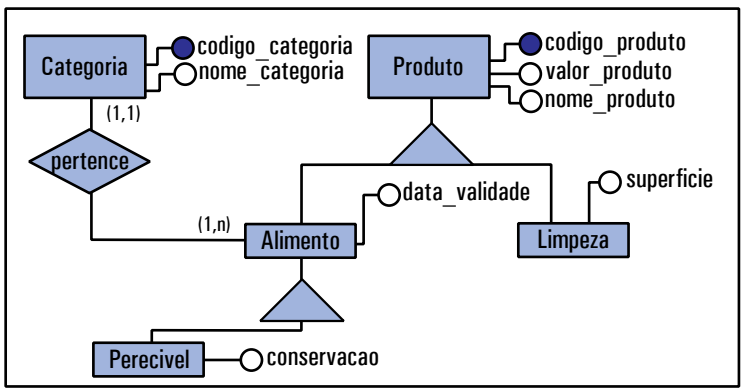
1) Explique os três modos de converter uma especialização para o modelo relacional.

2) Dê um exemplo de quando você aplicaria o primeiro modo de conversão da especialização.

3) Dê um exemplo de quando você aplicaria o segundo modo de conversão da especialização.

4) Dê um exemplo de quando você aplicaria o terceiro modo de conversão da especialização.

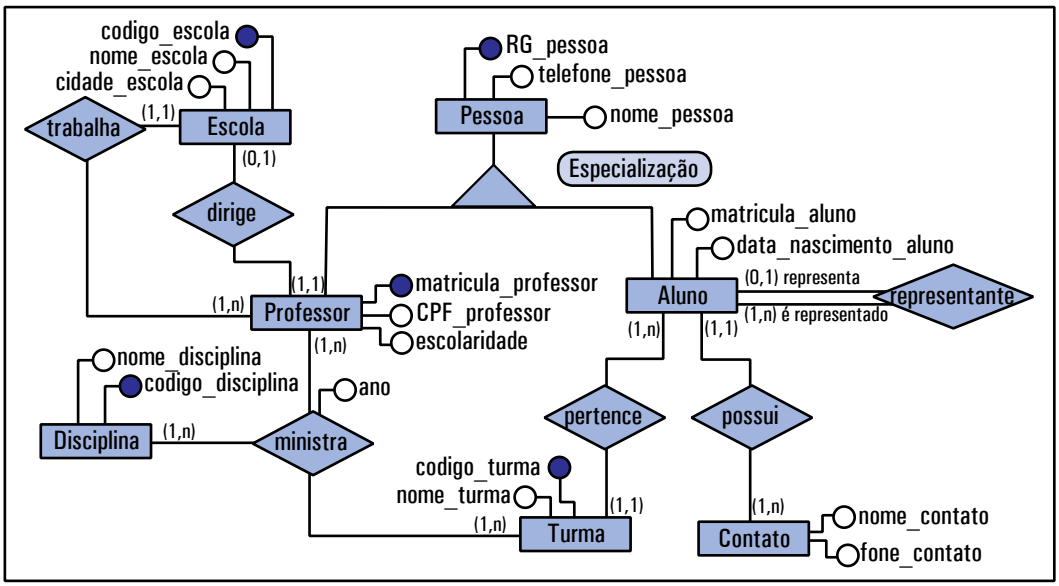
5) Passe o Diagrama de ER abaixo para o modelo relacional. Especifique os tipos de dados para cada atributo.



6) Desenhe o Diagrama do Modelo Relacional para o exercício 5.

7) Explique o que é cada uma das restrições para os atributos (Valor Nulo, Regra de Domínio, Chaves, Valor *Default* e *Unique*) apresentadas no Dicionário de Dados. Em seguida, compare as suas explicações com as que foram dadas no texto e verifique se estão corretas.

8) Passe o Diagrama de ER abaixo para o Diagrama do Modelo Relacional. Especifique os tipos de dados para cada atributo.



9) Verifique se as tabelas abaixo estão nas três formas normais. Se não estiverem, normalize-as e especifique qual forma normal está sendo aplicada.

a.

```
tbAcidente (numero_placa_carro, CPF_motorista,
nome_motorista, total_danos_acidente, data_acidente)
```

b.

```
tbPaciente(codigo_paciente, nome_paciente, (fone_paciente),
(CRM_medico, nome_medico, data_consulta), codigo_con-
venio, nome_convenio, (codigo_exame, nome_exame,
diagnostico_principal))
```

10) Normalize as seguintes tabelas de dados, especificando qual forma normal está sendo aplicada em cada passo da normalização. Em seguida, desenhe o diagrama ER correspondente ao esquema relacional obtido.

a.

```
tbAluno (cod_aluno, nome_aluno, sexo_aluno, data_nas-
cimento_aluno, codigo_curso, nome_curso, nome_diretor,
(codigo_disciplina, nome_disciplina, nota_disciplina))
```

b.

```
tbNotaFiscal (Numero_nota, data_emissao_nota, codigo_
cliente, nome_cliente, endereço_cliente: rua; numero;
e complemento, CPF_cliente, (codigo_produto, nome_pro-
duto, quantidade_vendida_produto, valor_unitario_produto,
valor_total_por_item_vendido, valor_total_da_nota))
```